OKE Network Design - Lab Guide

OKE cluster with a public Kubernetes API endpoint and private worker and pod networking. Region: ap-hyderabad-1. Address range used in this guide: 172.16.0.0/16.

How to use this guide: follow the steps in order in the OCI Console. Steps 1-4 include screenshots from your lab (12 figures). Dashed boxes mark screenshots still to capture (Internet Gateway, and Steps 5-10).

1. Design summary

Item	Design decision
Region	ap-hyderabad-1
Public subnet use	Kubernetes API endpoint (and optional load balancer)
Private subnet use	Worker node pool
Dedicated pod subnet	Required only for VCN-native pod networking
Internet Gateway	Only for the public API endpoint subnet (and optional LB subnet)
NAT and Service Gateway	Outbound access for private workers and pods
Flannel overlay	Pods do not need a separate VCN pod subnet

2. CIDR layout

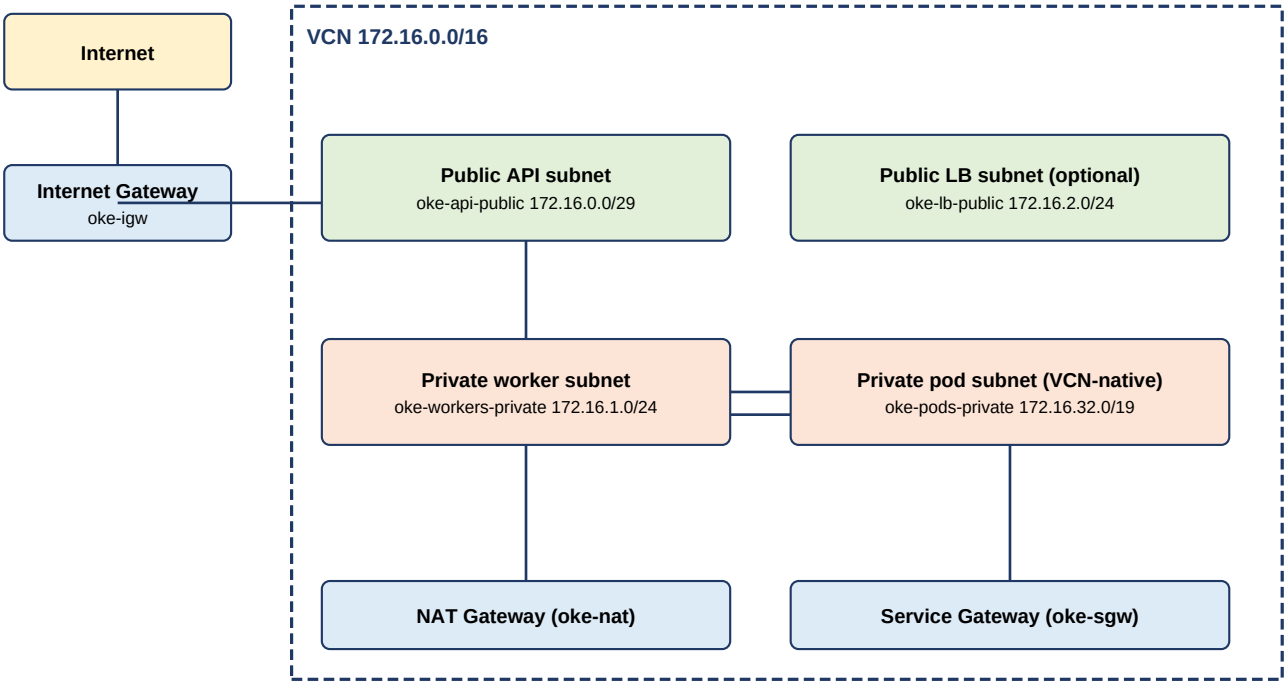
Resource	Name	CIDR	Type
VCN	oke-vcn	172.16.0.0/16	VCN
API endpoint subnet	oke-api-public	172.16.0.0/29	Public
Worker node subnet	oke-workers-private	172.16.1.0/24	Private
Load balancer subnet (optional)	oke-lb-public	172.16.2.0/24	Public
Pod subnet (VCN-native only)	oke-pods-private	172.16.32.0/19	Private

To use another private range (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16), change the first two octets everywhere. Avoid overlap with on-premises/VPN networks, 10.244.0.0/16 (Flannel), 10.96.0.0/16 (services) and 169.254.0.0/16. The companion spreadsheet recalculates all CIDRs from one input cell.

Names used in your screenshots

Guide name	Name in screenshots
oke-igw	igw
oke-nat	ngw
oke-sgw	sgw
oke-api-rt, oke-lb-rt	Public route table
oke-workers-rt, oke-pods-rt	Private route table
API endpoint security group	apiserver (security list)

3. Target topology



Solid lines show connections between components. Private subnets reach the internet only through the NAT Gateway and reach OCI services through the Service Gateway.

4. Console steps

Step 1: Create the VCN

- Open the OCI Console and select region ap-hyderabad-1.
- Networking > Virtual Cloud Networks, choose the correct compartment.
- Click Create VCN. Name: oke-vcn, CIDR: 172.16.0.0/16, enable DNS resolution. Create.



Figure 1: Create VCN - name oke-vcn, IPv4 CIDR 172.16.0.0/16.

Step 2: Create the gateways

- VCN details > Resources. Internet Gateway: create **oke-igw** (enabled).
- NAT Gateway: create **oke-nat**.
- Service Gateway: create **oke-sgw**, select All Services in Oracle Services Network.

SCREENSHOT PLACEHOLDER
Step 2 - Internet Gateway (igw)
(paste your OCI Console screenshot here)

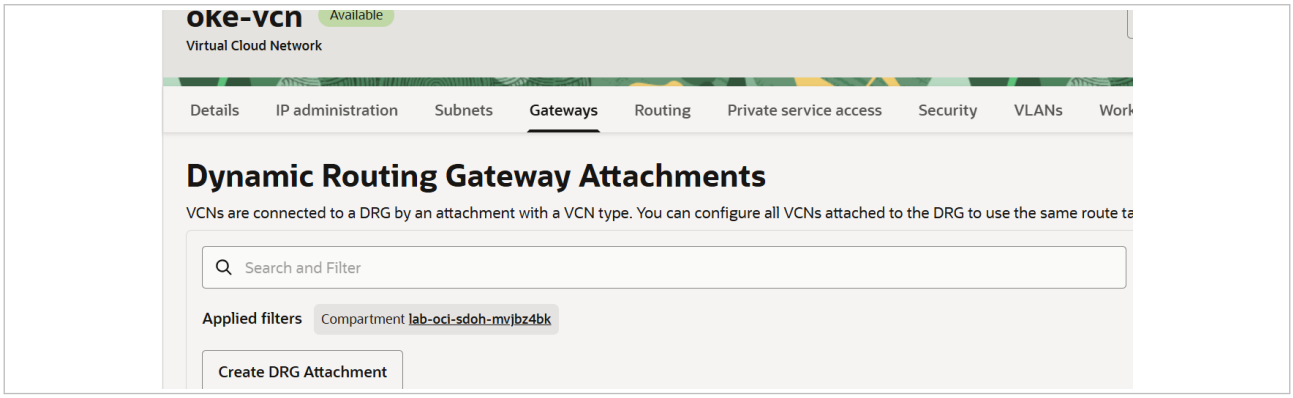


Figure 2: oke-vcn > Gateways tab.

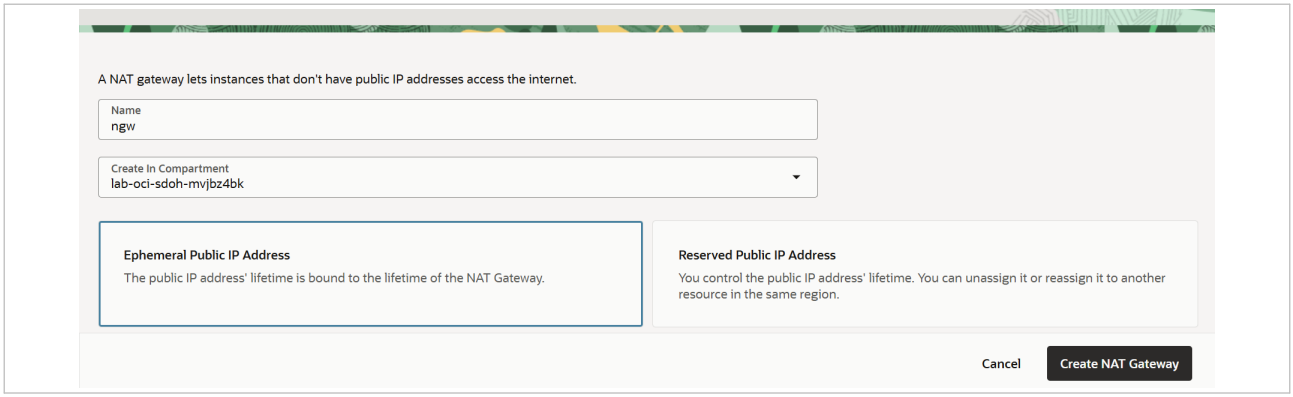


Figure 3: Create NAT Gateway - named ngw (guide name oke-nat).

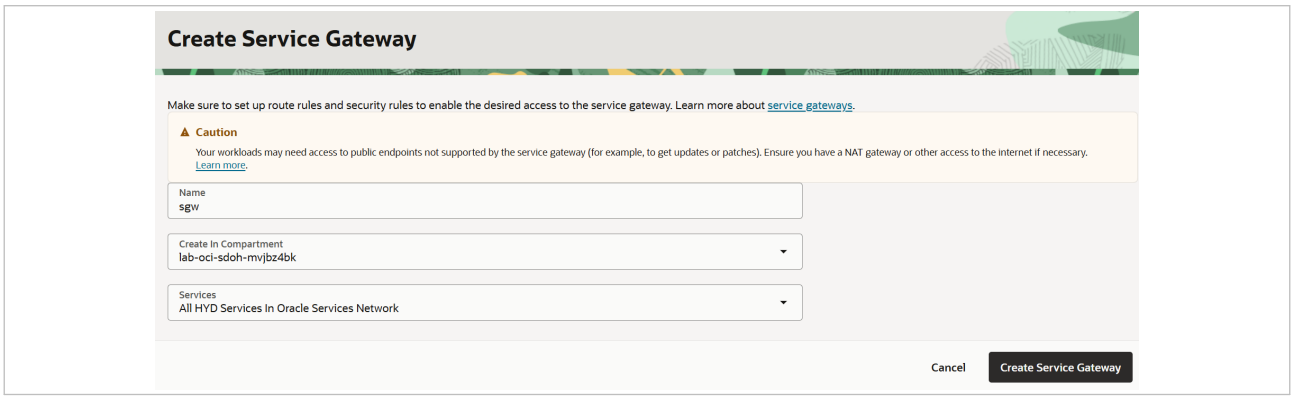


Figure 4: Create Service Gateway - named sgw (guide name oke-sgw), All HYD Services in Oracle Services Network.

Step 3: Create the route tables (before the subnets)

- VCN details > Routing > Create Route Table. Create the tables below first so each subnet can select its own table when it is created.
- **oke-api-rt**: rule 0.0.0.0/0 -> Internet Gateway (oke-igw).
- **oke-workers-rt**: rule 0.0.0.0/0 -> NAT Gateway (oke-nat); rule destination type Service, All <region> Services in Oracle Services Network -> Service Gateway (oke-sgw).
- **oke-pods-rt** (VCN-native only): same two rules as oke-workers-rt.
- **oke-lb-rt** (optional): rule 0.0.0.0/0 -> Internet Gateway (oke-igw).

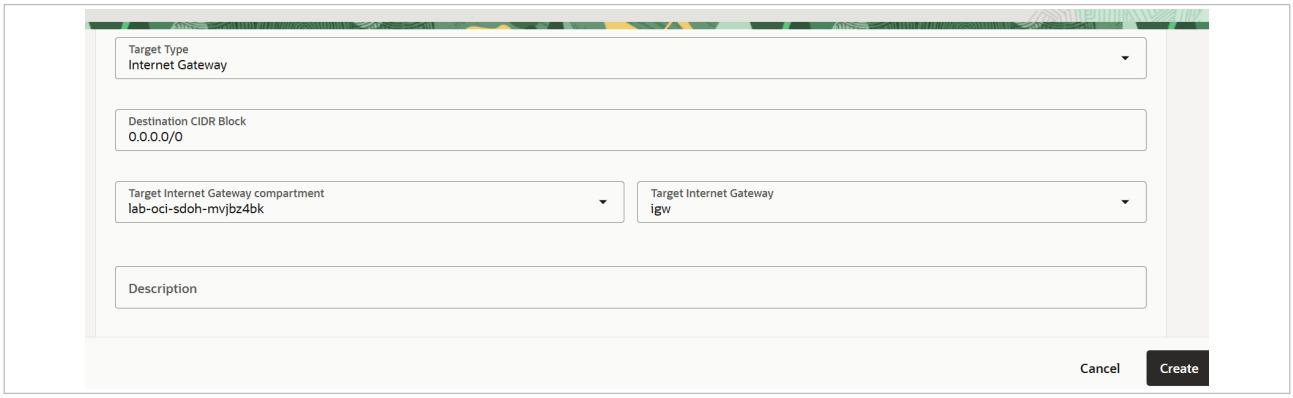


Figure 5: Public route table rule - Internet Gateway, 0.0.0.0/0, target igw.

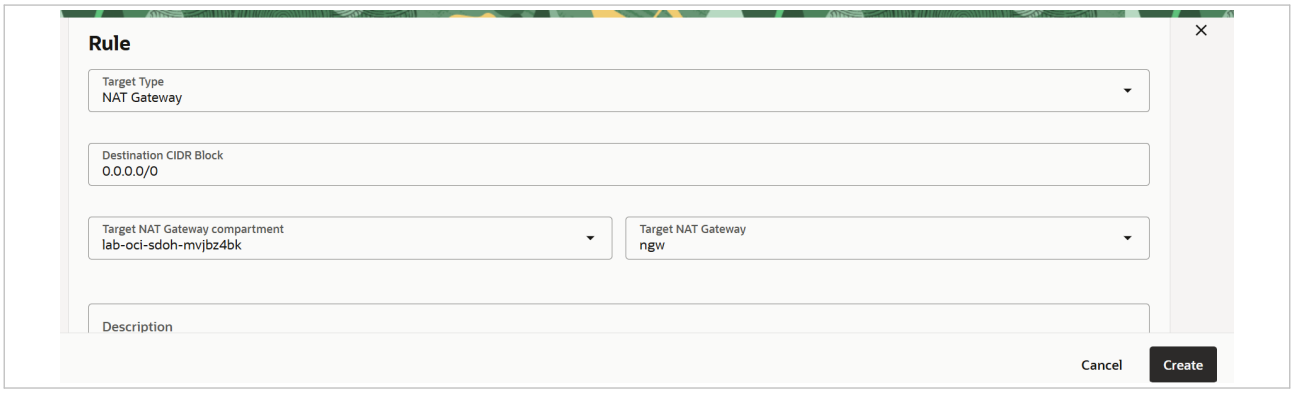


Figure 6: Private route table rule - NAT Gateway, 0.0.0.0/0, target ngw.

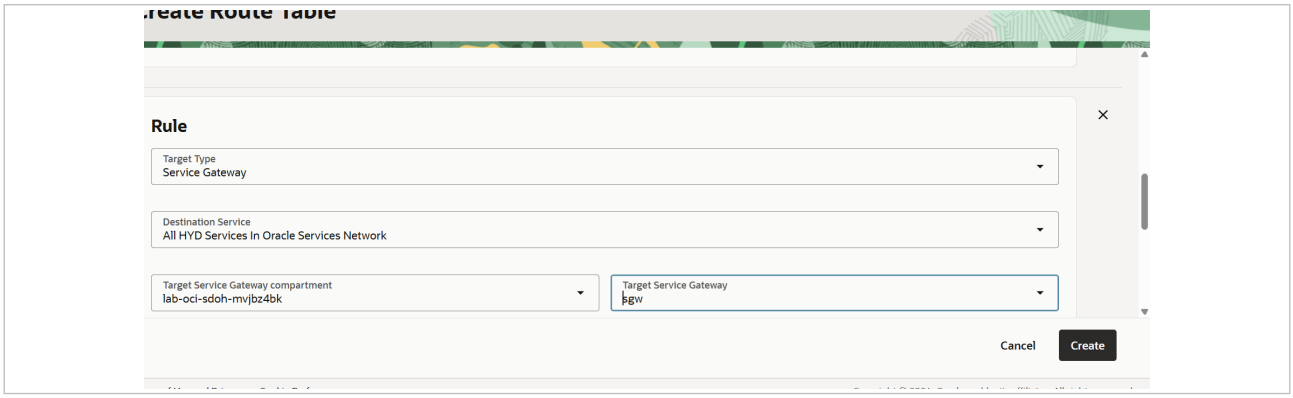


Figure 7: Private route table rule - Service Gateway, All HYD Services, target sgw.

Step 4: Create the security groups (NSGs) before the subnets

- VCN details > Security > Network Security Groups > Create Network Security Group. NSGs belong to the VCN, so they can exist before any subnet.
- Create **nsg-api** (Kubernetes API endpoint), **nsg-workers** (node worker group) and **nsg-lb** (load balancer, optional).
- VCN-native pod networking only: also create **nsg-pods**.
- Add the rules from Section 5 to each NSG. The CIDRs are already planned, so the rules can be written now. Rules are stateful; with stateless rules add the matching return rules.
- nsg-api includes ingress TCP 6443 from 0.0.0.0/0 for the public API endpoint (see Section 5.1).
- The screenshots below create **Security Lists** rather than NSGs. Both work: the same rules apply either way. Use one method consistently, and attach security lists to the subnets in Steps 5-7.
- Worker node ports (nsg-workers) are listed in the table below; the matching rules on nsg-api are ingress 6443 and 12250 from the worker subnet, and egress 10250 to it.
- Attach them later: nsg-api when creating the cluster, nsg-workers (and nsg-pods) on the node pool, nsg-lb on the load balancer.

nsg-workers port	Protocol	Direction	Peer	Purpose
6443	TCP	Egress	API endpoint 172.16.0.0/29	Kubelet / kube-proxy to Kubernetes API
12250	TCP	Egress	API endpoint 172.16.0.0/29	Workers to OKE control plane (not 12251)
10250	TCP	Ingress	API endpoint 172.16.0.0/29	API server to kubelet (logs, exec, metrics)
6443, 12250	TCP	Ingress	API endpoint 172.16.0.0/29	Control plane to workers where required
22	TCP	Ingress	ADMIN_CIDR / bastion	SSH to workers (restricted)
30000-32767, 10256	TCP	Ingress	LB 172.16.2.0/24 (opt.)	NodePort range and kube-proxy health check
All	All	In / Out	Workers 172.16.1.0/24, Pods 172.16.32.0/19	Node-to-node and pod traffic
443, 80	TCP	Egress	OCI Services / Internet (NAT)	Image pulls, updates

Rules to enter with the 172.16 ranges. Figures 11-12 now show these same values (last column).

List	Dir	Source	Protocol	Port	Screenshot shows
apiserver	In	172.16.0.0/29	TCP	6443, 10250, 12250	172.16.0.0/29 (already correct)
API endpoint	In	0.0.0.0/0	TCP	6443	0.0.0.0/0 (public API, lab)
API endpoint	In	172.16.1.0/24	TCP	6443	172.16.1.0/24
API endpoint	In	172.16.1.0/24	TCP	12250	172.16.1.0/24
API endpoint	In	172.16.1.0/24	ICMP	Type 3, Code 4	172.16.1.0/24
Workers	In	172.16.1.0/24	All	All	172.16.1.0/24
Workers	In	172.16.0.0/29	ICMP	Type 3, Code 4	172.16.0.0/29
Workers	In	172.16.0.0/29	TCP	All	172.16.0.0/29
Workers	In	0.0.0.0/0	TCP	22	0.0.0.0/0 (restrict to ADMIN_CIDR for production)

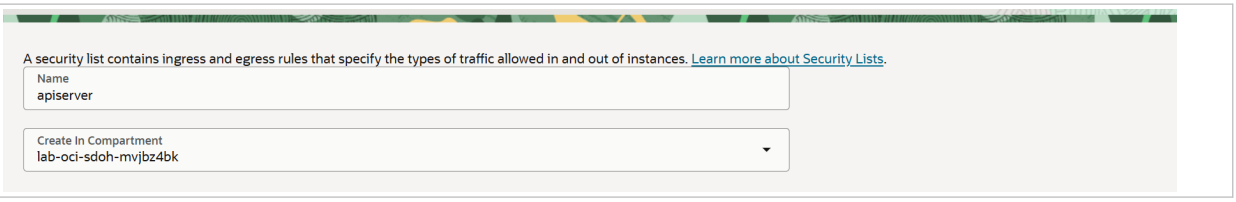


Figure 8: Create Security List - named apiserver.

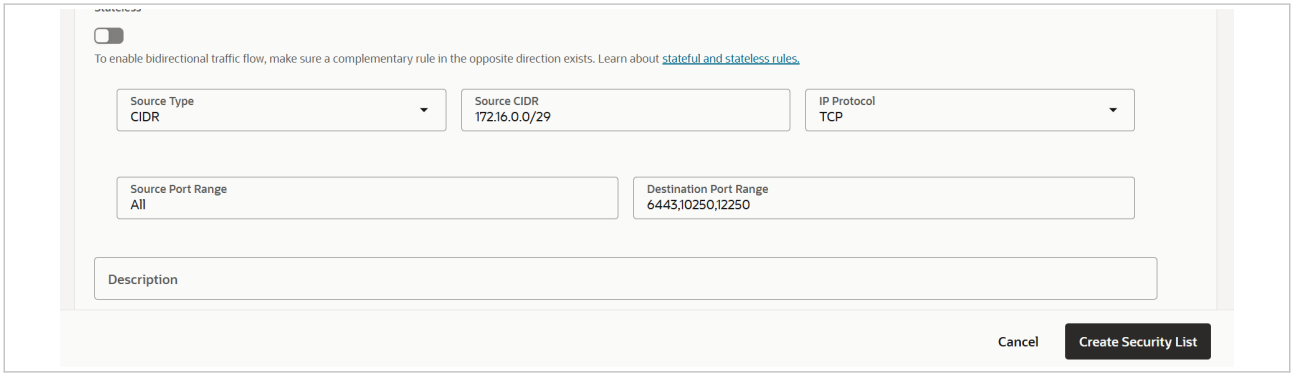


Figure 9: Ingress rule - source 172.16.0.0/29, TCP, ports 6443,10250,12250.

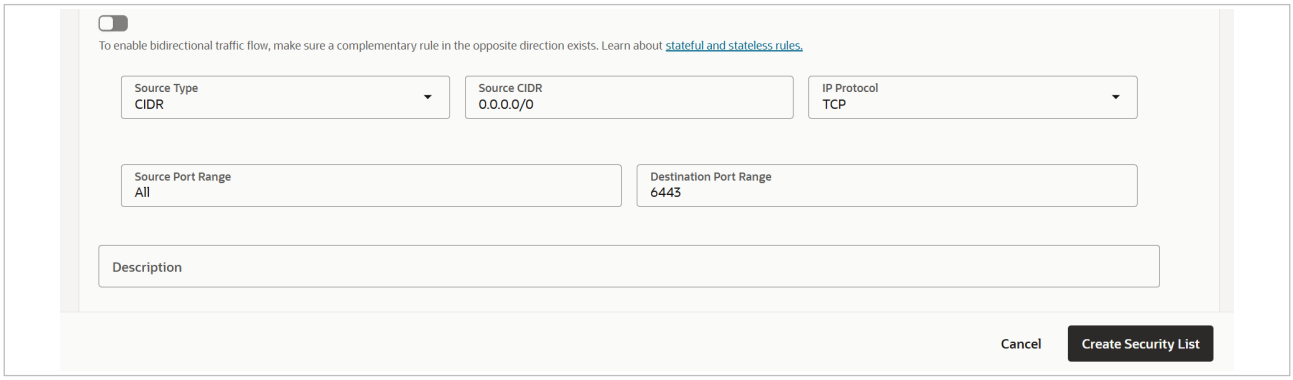


Figure 10: Ingress rule - source 0.0.0.0/0, TCP 6443 (public API endpoint).

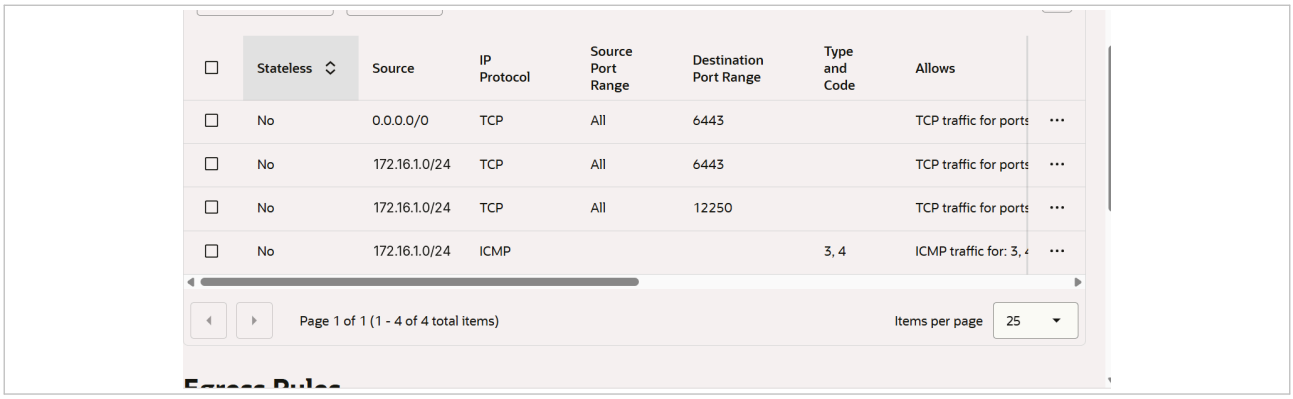


Figure 11: API endpoint rules list - sources 0.0.0.0/0 and 172.16.1.0/24.

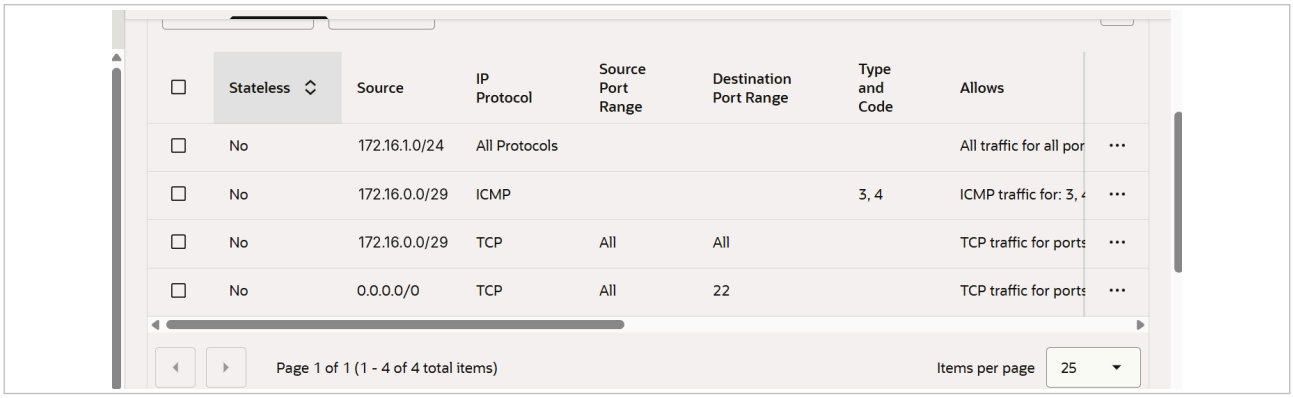
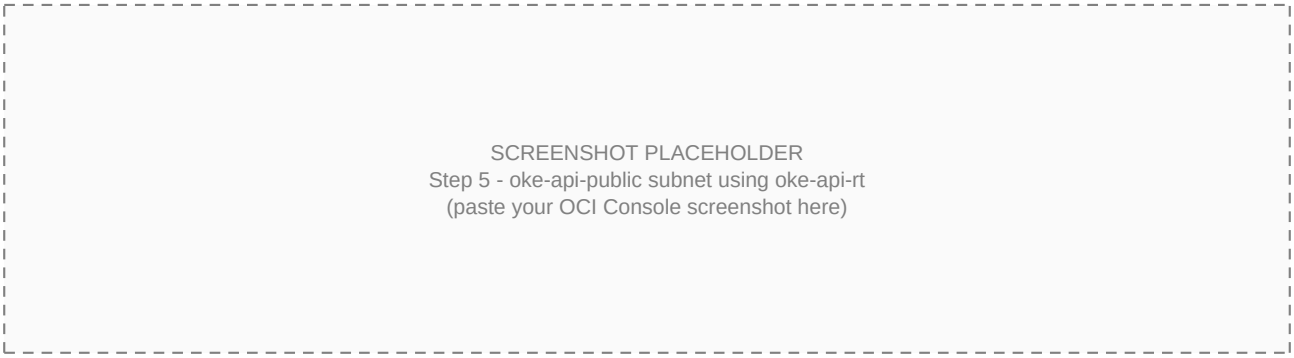


Figure 12: Worker rules list - sources 172.16.1.0/24, 172.16.0.0/29 and 0.0.0.0/0.

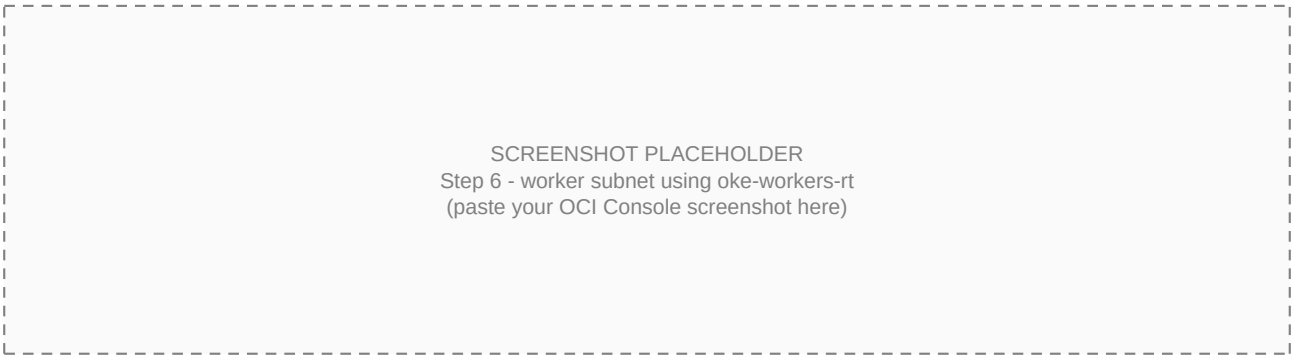
Step 5: Create the public API endpoint subnet

- Create **oke-api-public**, Regional, CIDR 172.16.0.0/29, public subnet.
- Route table: select **oke-api-rt** (0.0.0.0/0 -> oke-igw).
- Public endpoint: nsg-api allows TCP **6443** from **0.0.0.0/0** so kubectl works from anywhere. Warning: this exposes the API server to the internet; for production restrict it to ADMIN_CIDR.
- If the subnet still uses its default security list, make sure TCP 6443 is allowed there as well, since the security list and NSG rules both apply.



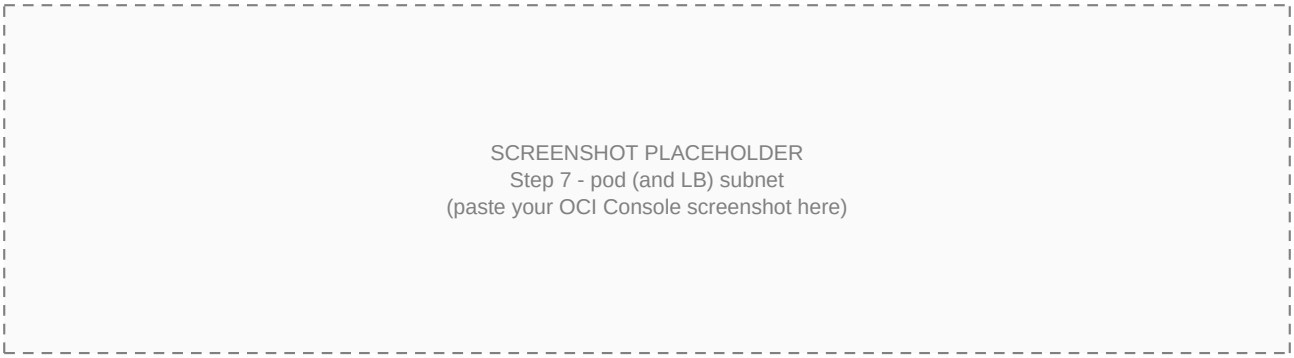
Step 6: Create the private worker subnet

- Create **oke-workers-private**, Regional, CIDR 172.16.1.0/24, private (no public IPs).
- Route table: select **oke-workers-rt** (NAT Gateway + Service Gateway rules).



Step 7: Create the private pod subnet (VCN-native only)

- Create **oke-pods-private**, CIDR 172.16.32.0/19, private.
- Route table: select **oke-pods-rt** (NAT Gateway + Service Gateway rules).
- Ensure no overlap with other subnets, the Kubernetes service CIDR, or connected on-premises/VPN networks.
- Optional: create **oke-lb-public** (172.16.2.0/24), public, route table **oke-lb-rt**, only if you use LoadBalancer Services.



Step 8: Create the OKE cluster

- Developer Services > Kubernetes Clusters (OKE) > Create Cluster > Custom Create.
- Select oke-vcn; Kubernetes API endpoint subnet: oke-api-public; enable public IP; attach nsg-api.
- Networking type: VCN-native pod networking (dedicated pod subnet) or Flannel overlay (no pod subnet).
- Worker subnet: oke-workers-private; pod subnet: oke-pods-private (VCN-native only). Create the cluster.

SCREENSHOT PLACEHOLDER
Step 8 - cluster create review page
(paste your OCI Console screenshot here)

Step 9: Create the node pool

- Cluster > Node pools > Create node pool.
- Subnet oke-workers-private, private IP addresses only.
- Choose shape, image, Kubernetes version and node count. Attach nsg-workers (and nsg-pods for VCN-native). Create.

SCREENSHOT PLACEHOLDER
Step 9 - node pool active
(paste your OCI Console screenshot here)

Step 10: Verify

- Cluster state Active; nodes Ready.
- Cluster > Access cluster, run the kubeconfig command, then `kubectl get nodes`.

SCREENSHOT PLACEHOLDER
Step 10 - kubectl get nodes output
(paste your OCI Console screenshot here)

Important: the Internet Gateway serves the public API (and optional LB) subnet only. Private worker and pod subnets use the NAT Gateway for internet and the Service Gateway for OCI services.

5. Security rules (NSG)

Replace ADMIN_CIDR with your office, VPN, bastion or CI/CD public CIDR. The public API endpoint rule allows TCP 6443 from 0.0.0.0/0 as requested; this exposes the API server to the internet (kubectl still needs valid credentials), so for production remove it and keep only ADMIN_CIDR. Never use 0.0.0.0/0 for TCP 22.

5.1 nsg-api

Dir	Source / Dest	Protocol	Port	Purpose
In	Workers 172.16.1.0/24	TCP	6443	Workers to API
In	Workers 172.16.1.0/24	TCP	12250	Workers to control plane
In	Pods 172.16.32.0/19	TCP	6443, 12250	Pods to API (VCN-native)
In	ADMIN_CIDR	TCP	6443	kubectl, CI/CD, VPN, bastion
In	0.0.0.0/0 (public)	TCP	6443	Public API endpoint access from anywhere (lab; see warning)
In	Workers 172.16.1.0/24	ICMP	Type 3, Code 4	Path MTU discovery
Out	OCI Services (Service GW)	TCP	443	OCI service access
Out	Workers 172.16.1.0/24	TCP	10250	API to kubelet
Out	Pods 172.16.32.0/19	TCP	All	API to pods (webhooks)
Out	Workers 172.16.1.0/24	ICMP	Type 3, Code 4	Path MTU discovery

5.2 nsg-workers

Dir	Source / Dest	Protocol	Port	Purpose
In	Workers 172.16.1.0/24	All	All	Node-to-node (incl. Flannel UDP 8472)
In	Pods 172.16.32.0/19	All	All	Pod-to-worker
In	API 172.16.0.0/29	TCP	10250	Kubelet API from control plane
In	API 172.16.0.0/29	TCP	6443, 12250	Control plane to workers
In	ADMIN_CIDR / bastion	TCP	22	SSH (restricted)
In	LB 172.16.2.0/24 (opt.)	TCP	30000-32767	NodePort from LB
In	LB 172.16.2.0/24 (opt.)	TCP	10256	kube-proxy health check
In	0.0.0.0/0	ICMP	Type 3, Code 4	Path MTU discovery
Out	Workers 172.16.1.0/24	All	All	Node-to-node
Out	Pods 172.16.32.0/19	All	All	Worker-to-pod
Out	API 172.16.0.0/29	TCP	6443, 12250	Kubelet/kube-proxy to API
Out	API 172.16.0.0/29	ICMP	Type 3, Code 4	Path MTU discovery
Out	OCI Services (Service GW)	TCP	443	OCIR image pulls etc.
Out	0.0.0.0/0 (NAT)	TCP	443, 80	Registries, updates
Out	0.0.0.0/0 (NAT)	UDP/TCP	53, 123	DNS and NTP

5.3 nsg-pods (VCN-native only)

Dir	Source / Dest	Protocol	Port	Purpose
In	Pods 172.16.32.0/19	All	All	Pod-to-pod
In	Workers 172.16.1.0/24	All	All	Worker-to-pod
In	API 172.16.0.0/29	TCP	All	API server to pods (webhooks)
In	LB 172.16.2.0/24 (opt.)	TCP	Pod service ports	LB to pods
In	0.0.0.0/0	ICMP	Type 3, Code 4	Path MTU discovery

Dir	Source / Dest	Protocol	Port	Purpose
Out	Pods 172.16.32.0/19	All	All	Pod-to-pod
Out	Workers 172.16.1.0/24	All	All	Pod-to-worker
Out	API 172.16.0.0/29	TCP	6443, 12250	Pods to API/control plane
Out	OCI Services (Service GW)	TCP	443	OCI services
Out	0.0.0.0/0 (NAT)	TCP	443, 80	Outbound internet

5.4 nsg-lb (optional)

Dir	Source / Dest	Protocol	Port	Purpose
In	0.0.0.0/0 (or restricted)	TCP	443, 80	Public HTTPS/HTTP
Out	Workers 172.16.1.0/24	TCP	30000-32767, 10256	NodePort and health checks
Out	Pods 172.16.32.0/19	TCP	Pod service ports	Direct to pods (VCN-native)

5.5 Quick port reference

Port	Protocol	Used for
22	TCP	SSH to workers (admin/bastion only)
443	TCP	HTTPS: OCI services, registries, public ingress
6443	TCP	Kubernetes API server
12250	TCP	OKE control plane (kubelet / proxymux)
10250	TCP	Kubelet API
10256	TCP	kube-proxy health check
30000-32767	TCP	NodePort range
8472	UDP	Flannel VXLAN overlay
ICMP 3,4	ICMP	Path MTU discovery

With Flannel overlay, skip the pod subnet and nsg-pods. Verify final rules against Oracle's OKE network configuration documentation before production use.